Программирование на Python

Лекция 1

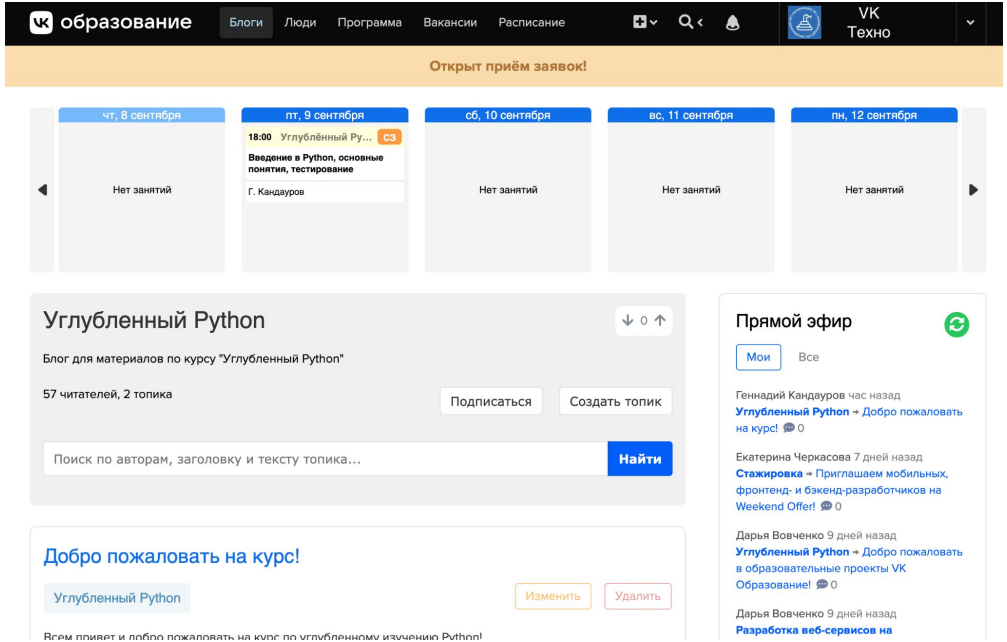
Введение, основные понятия

Кандауров Геннадий

education

Напоминание отметиться на портале

+ оставить отзыв



План курса

1. Введение в Python
2. Функции и методы
3. Функции. Продолжение
4. Введение в классы и ООП
5. Генераторы и итераторы
6. Работа с файлами
7. РК
8. Стандартная библиотека
9. Pandas и numpy
10. Pandas и numpy - 2
11. Параллелизм в Python
12. Профилирование, отладка, логирование
13. Алгоритмы в Python
14. Экзамен/пересдача

Контроль

- **Домашние задания (ДЗ)** - 60 баллов
- **Квизы** - 10 баллов в домашки

  можно получить один балл за все правильные ответы в квизе
- **РК** - 10 баллов
- **Экзамен** - 30 баллов

В этом квизе можно набрать 14 баллов
У меня 1 ошибка. Мне ведь дадут дополнительный балл?
Так ведь?

Домашние задания

- **десять** ДЗ
- **шесть** баллов до мягкого дедлайна
- **три** балла после мягкого дедлайна
- мягкий дедлайн - **две недели** после выдачи ДЗ, указан на портале
- **три** попытки, в зачет идет лучшая
- без списываний - баллы за домашку будут обнуляться

На оценку влияют:

1. Соответствие решения заданию
2. Корректность решения
3. Оформление решения
4. Код решения как таковой
5. Дедлайны

Домашние задания

Сдача ДЗ:

- репозиторий должен быть приватным
- ветка main
- каждое ДЗ располагается в каталоге по номеру ДЗ без префиксов или суффиксов: "**01**", "**02**", ... "**10**"

  например, каталог для первой домашки должен называться "**01**";
- вспомогательные каталоги внутри каталога решения (**01**, **02**) не нужны, то есть все файлы и модули одной домашки должны находиться в корневом каталоге этой домашки (**01**, **02**);

Конкурс мемов

Я: Мам, а можно мне ML в продакшн?

Мама: Нет, у нас есть ML в продакшн дома!

ML в продакшн дома:

3. Тесты в отдельном модуле для каждого пункта

```python
class SomeModel:
    def predict(self, message: str) -> float
        # реализация не важна


def predict_message_mood(
    message: str,
    model: SomeModel,
    bad_thresholds: float = 0.3,
    good_thresholds: float = 0.8,
) -> str:
    ...
```

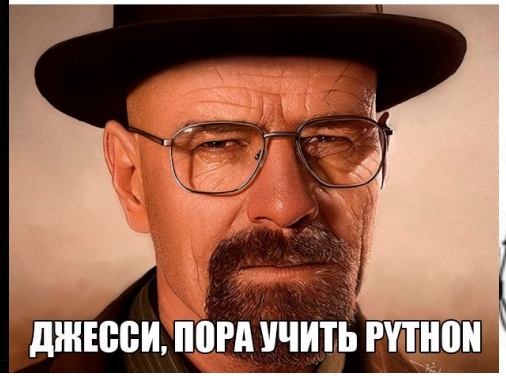
```python
class ClassMeth:
    name = "cls_name"

    @classmethod
    def init_42(cls):
        print(f"Class {cls=}")

    def __init__(self, val):
        self.val = val

    def inst_meth(self):
        print(f"Instance: {self.val=}")

ClassMeth(42)
```

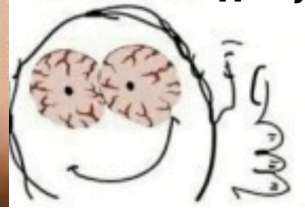


Как спят студенты?

Как спят преподаватели?

А НИКАК

До 4 утра дз проверяют



Содержание занятия

1. Основные понятия
2. Типы данных
3. Управляющие конструкции
4. Функции

Python

Python

Интерпретируемый язык с динамической и утиной типизацией, автоматической сборкой мусора и GIL.

Python - название спецификации языка

Реализации:

- CPython
- IronPython
- Jython
- PyPy

https://github.com/python/cpython

https://www.python.org/doc/

Дзен Python

```python
import this
```

```
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

Все есть объект

Objects are Python's abstraction for data. All data in a Python program is represented by objects or by relations between objects.

docs.python.org

PyObject

```
typedef struct _object {
    _PyObject_HEAD_EXTRA
    Py_ssize_t ob_refcnt;
    PyTypeObject *ob_type;
} PyObject;
```

Объект

1. Каждый объект имеет идентичность, тип, значение
2. **`id`** никогда не меняется после создания объекта (**`is`** сравнивает **`id`** объектов)
3. Тип объекта определяет какие операции с ним можно выполнять
4. Значение объекта может меняться

Переменные

```
num = 1  # операция присваивания переменной num значения 1
```

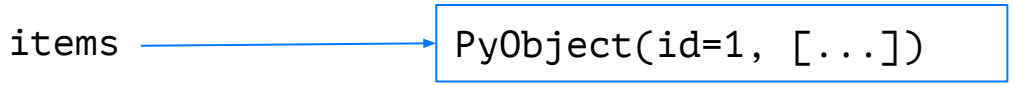
- допустимые символы: буквы, цифры, _ (подчеркивание)
- начинаться только с буквы или _

```
# valid
num = 1
_num = 1
__num = 1
число1_2_34 = 1
_123_число = 1
```

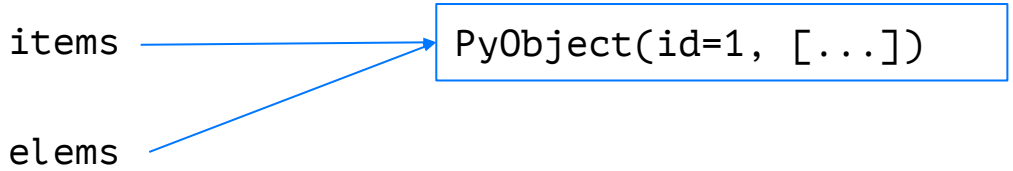
```
# invalid
1num = 1
1_num = 1
```

Переменные

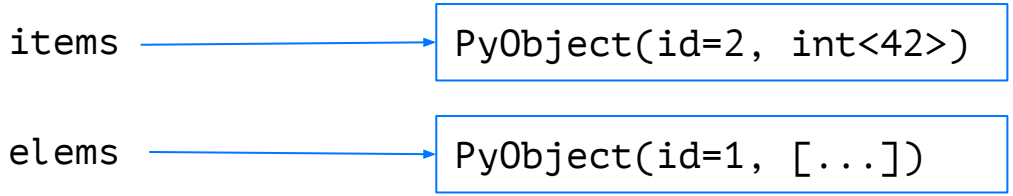
```
items = [1, 2, 3]
```




```
elems = items
```




```
items = 42
```



Базовые типы: типы с одним значением

- `None`
- `NotImplemented`
- `Ellipsis (...)`

Изменяемые, неизменяемые типы

Неизменяемые

- `bool`
- `int, float, complex`
- `str, bytes`
- `tuple`
- `frozenset`

Изменяемые

- `list`
- `dict, set`
- `user defined`

Базовые типы: числа

Целые числа (`int`)

```
num = 42
num = 42_000_000
num = 0o10  # 8
num = 0x10  # 16
```

Вещественные числа (`float`)

```
num = 3.14
```

Комплексные числа (`complex`)

```
num = 9 + 0.75j  # num.real, num.imag
```

Арифметические операции

- Сложение: +
- Вычитание: -
- Умножение: *
- Деление: /
- Целочисленное деление: //
- Остаток от деления: %
- Возведение в степень: **

Битовые операции

- Умножение: &
- Сложение: |
- Инверсия: ~
- Исключающее ИЛИ: ^
- Побитовый сдвиг: <<, >>

Базовые типы: bool

Логический тип (`bool`)

```
is_true = True  # или False
```

- Логическое И: **`and`**
- Логическое ИЛИ: **`or`**
- Отрицание: **`not`**

Вычисление составных логических выражений **ленивое:**

```
return_false() and return_true()
```

```
return_true() or return_false()
```

Базовые типы: строки

```
s = "просто строка"  # str
byte_s = b"qwerty"  # bytes
raw_s = r"111\nданные"  # str

str.encode() -> bytes
bytes.decode() -> str

str1 + str2  # конкатенация
"a" * 5  # повторение "aaaaa"
```

Форматирование

- `"x %s %d" % ("y", 42)`
- `"x %(y)s %(n)d" % dict(y="y", n=42)`
- `"x {} {}".format("y", 42)`
- `y, n = "y", 42`

  `f"x {y} {n}"`

"x y 42"

Базовые типы: коллекции

Список (list)

```
lst = []
lst = [1, 2, 3]
lst1 = list(lst)
lst.append(4)
```

Кортеж (tuple)

```
tup = ()
tup = (1,)
tup = 1, 2, 3
tup = tuple(lst)
```

Swap

```
a, b = b, a
```

Распаковка

```
lst = [1, 2, 3, 4, 5]
a, b, *c = lst  # 1, 2, [3, 4, 5]
a, *b, c = lst  # 1, [2, 3, 4], 5

lst = ["123", "456"]
(a, *b), c = lst
# "1", ["2", "3"], "456"
```

Базовые типы: коллекции

Словарь (`dict`)

```
d = {}  # dict()
d1 = {1: 11}
d1[2] = 22
d2 = dict(x=10, y=20)
d3 = {**d1, **d2, 3: 33}
```

Множество (`set`, `frozenset`) уникальных неизменяемых объектов.
Множество не индексируется, но по нему можно итерироваться.

```
s = set()
s = {1, 2, 3}
s.add(4)
s = set("123123123")  # {"1", "2", "3"}
```

Базовые типы: коллекции

```
lst = [1, 3, 2]
dct = {1: 11, 2: 22}
```

- Оператор in:
  ```
  1 in lst
  1 in dct
  "456" in "123456"
  ```
- Сортировка:
  ```
  sorted(dct)
  lst.sort()
  ```
- Итерирование:
  ```
  for key in dct: pass
  ```

Списковые включения (компрехеншены)

```
lst = [n ** 2 for n in range(10) if n % 2]

s = {ch for ch in "abcabcbca"}

dct = {n: n ** 2 for n in range(10) if n % 2}

gen = (n ** 2 for n in range(10) if n % 2)
```

Управляющие конструкции: условный оператор

```
if cond1:
    action1()
elif cond2:
    action2()
else:
    action3()
```

Тернарный оператор

```
result = action1() if cond else action2()
```

Pattern matching (3.10)

```
value = 42
match value:
    case 41:
        act_41()
    case 42:
        act_42()
    case other:
        act_other(other)
```

Управляющие конструкции: циклы

Цикл `for`

```
for elem in sequence:
    process(elem)
    # break
    # continue
else:
    do_else()
```

Цикл `while`

```
while condition:
    process()
    # break
    # continue
else:
    do_else()
```

Управляющие конструкции: исключения

```
try:
    something_dangerous()
except DangerError as err:
    process_danger_error(err)
except TrivialError as err:
    process_trivial_error(err)
except Exception as err:
    process_total_error(err)
else:
    process_no_error()
finally:
    will_be_executed_in_any_case()
```

Управляющие конструкции: исключения

```
# правильно наследоваться от Exception
class TrivialError(Exception): pass

class DangerError(TrivialError): pass

# только когда абсолютно точно надо
class UserError(BaseException): pass

# совсем плохо
try:
    pass
except:
    process_err()
```

Управляющие конструкции: контекстный менеджер

```
class CtxManager:
    def __init__(self, name):
        self.db = connect_db(name)
    def __enter__(self):
        return self.db
    def __exit__(self, exc_type, exc_val, exc_tb):
        self.db.close()

# from contextlib import contextmanager

with CtxManager("db_name") as db:
    do_action(db)
```

Функции, lambda-функции

```
def add(a, b):
    return a + b

def do_action(action, *args, **kwargs):
    print(f"do {action} with {args}, {kwargs}")

    if callable(action):
        return action(*args, **kwargs)
    else:
        return action

do_action(add, 1, b=2)
do_action(lambda x, y: x * y, 5, y=6)
```

Классы

```
class A:
    cls_attr = 42

    def __init__(self, val):
        self.val = val
        self._protected = 10
        self.__private_val = 20

    def print_name(self):
        print(self.__class__.__name__)

    @property
    def private_val(self):
        return self.__private_val
```

```
class B(A):
    cls_attr = 55

    def __init__(self, bval, val):
        super().__init__(val)

        self.bval = val
        self.__private_val = 999


b = B(1, 2)
print(b.private_val)  # ???

isinstance(b, (int, A))  # ???
issubclass(B, object)  # ???
```

Модули и пакеты

Модули являются основным компонентом организации кода в питоне (и это тоже объекты), объединяются в пакеты.

```
import this
collections = importlib.import_module('collections')
itertools = __import__('itertools')
```

```
file.py - модуль
--app/ - пакет
     |--__init__.py
     |--hello.py
```

Валидные импорты

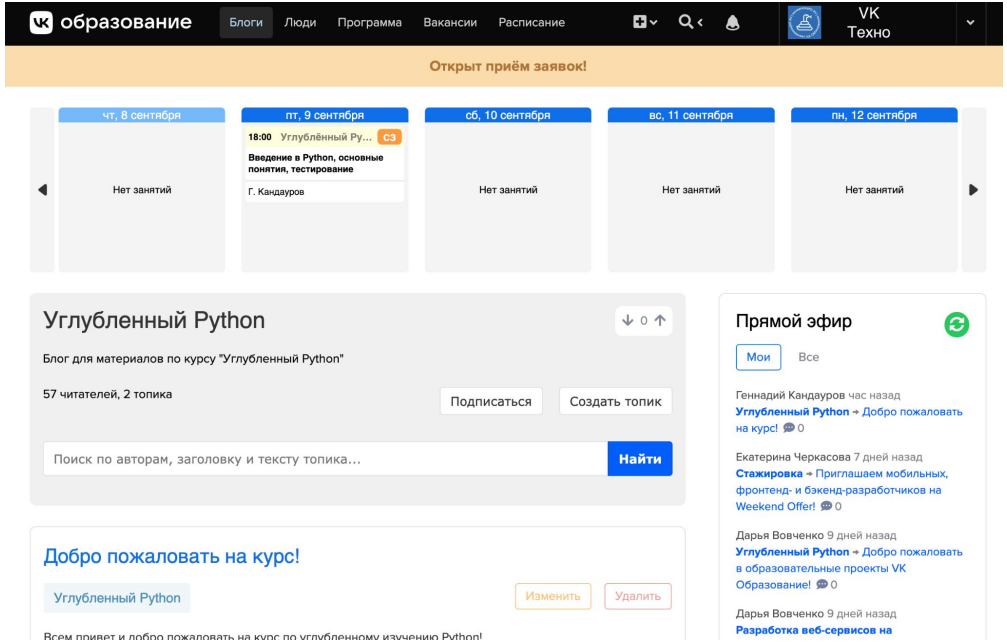
```
import sys
from sys import path
from sys import *
import sys as s
from sys import path as sys_path
```

Домашнее задание #01

- Реализация функций из задания

Напоминание отметиться на портале Vol 2

+ оставить отзыв после лекции



Спасибо за внимание

education